WEEK 1 · LECTURE 2

main(), warnings, and the safety net

Everything so far happened before your program ran. Now: what happens the moment it does.

MAIN, REVISITED

Return codes: how a process talks to its parent

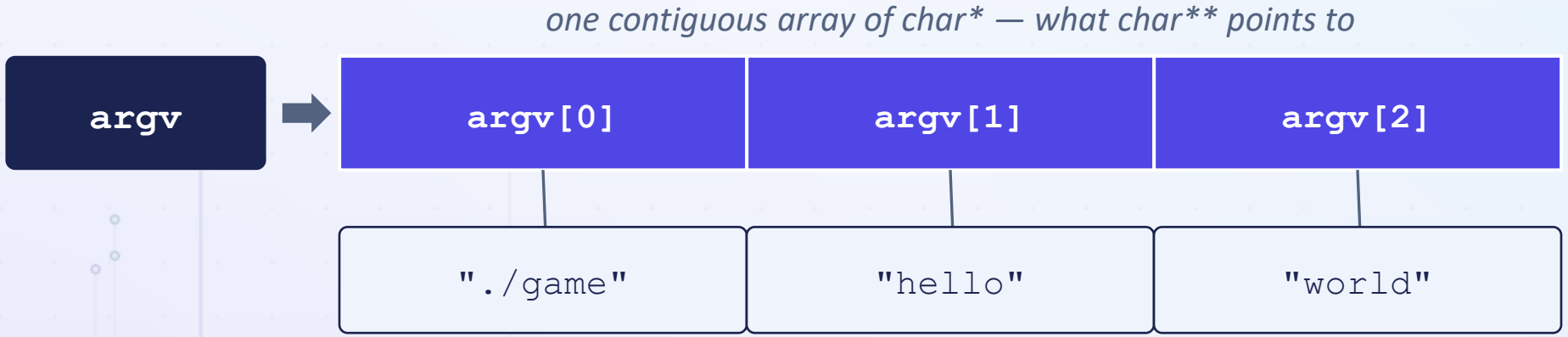
```
$ ./build/debug/bin/game
$ echo $?
0

# every process on the OS reports
# success/failure exactly this way
```

- int main() returns an exit code to the shell that launched it.
- 0 conventionally means success.
- Not C++-specific — ls, git, and the compiler itself all use this exact mechanism.

argc and argv

```
int main(int argc, char** argv)
```



*argv is a char** — a pointer to pointers to characters. You just used a pointer, successfully, weeks before the word was formal.*

READ THIS BEFORE YOU PANIC

Anatomy of a compiler warning

```
warn.cpp: In function 'int main()':
warn.cpp:2:9: warning: unused variable 'oops' [-Wunused-variable]
    2 |     int oops;
      |         ^~~~
```

File : line : column

warn.cpp:2:9 — go here first

Category

[-Wunused-variable] -- a searchable name

The caret

Points at the exact token the compiler means

THIS COURSE'S POLICY

debug shows it. strict fails on it.

`cmake --preset debug`

- Sanitizers on.
- Warnings are shown, but the build still succeeds.
- Use this for all day-to-day coursework.

`cmake --preset strict`

- Adds -Werror: every warning becomes a hard error.
- This is exactly what the grading server runs.
- Zero warnings, every submission, no exceptions.

WORDS WORTH KNOWING

Where “bug” and “debug” come from

September 9, 1947 — Harvard Mark II
Grace Hopper’s team found a real moth jamming a relay. They taped it into the logbook: “First actual case of bug being found.”
Not the word’s origin — but the moment it became irresistible.

- Engineers were calling technical faults “bugs” a century before computers existed.
- Thomas Edison complained about “bugs” in his own inventions in an 1878 letter.
- The word was already ordinary engineering slang by the time computers showed up.

BEFORE YOU DEBUG ANYTHING

Compile-time errors vs. runtime errors

Compile-time

- The compiler catches these before your program ever runs.
- Syntax mistakes, type mismatches, undeclared names, linker errors.
- The build simply fails — you get a message, not a program.

Runtime

- The compiler is satisfied. The program builds and runs.
- It crashes, hangs, or — worse — runs fine and prints a wrong answer.
- The compiler cannot see this. This is exactly what a debugger is for.

THE RIGHT MINDSET

Debugging is not whack-a-mole

A program covered in red squiggles, and a program that is silently wrong, are both broken.
Only one of them looks broken.

- The goal is a program that does the RIGHT thing — not one that merely stops complaining.
- Fixing messages one at a time, top to bottom, without asking why, is how new bugs get introduced.
- Form a hypothesis. Test it. Fix the actual cause. Then re-check the WHOLE program, not just the one symptom.

HANDS-ON, STARTING NOW

Setting a breakpoint in VS Code

```
// src/main.cpp

if (key == 'q') break;    // <<<
BREAKPOINT HERE

// click the gutter to the left of this
line,
// then press F5
```

- Click in the gutter, just left of the line number — a red dot appears.
- Or place your cursor on the line and press F9.
- Press F5 to launch. Execution runs at full speed until it hits your breakpoint, then freezes.

WHAT YOU'LL SEE WHEN IT STOPS

Three panels, one paused moment

1

Variables

Every local variable's current value, live — updates as you step.

2

Call Stack

Which function called which, all the way back to main.

3

The yellow line

Execution is paused here — before this line has run.

ONE LINE AT A TIME

Stepping through code

```
F5            continue
F10           step over
F11           step into
Shift + F11   step out

// same keys in CodeLLDB and
// other debug-adapter extensions
```

- Continue (F5) — run at full speed to the next breakpoint.
- Step Over (F10) — run this line without entering any function it calls.
- Step Into (F11) — follow execution into a function call.
- Step Out (Shift+F11) — finish the current function, return to its caller.

BRIDGING TO NEXT WEEK

Variables and console I/O, live

```
int items;
double price;

std::cout << "Items: ";
std::cin >> items;
std::cout << "Price each: ";
std::cin >> price;

double total = items * price;
std::cout << "Total: $" << total;
```

- Every variable needs a type, declared before its first use.
- std::cin reads from the keyboard; std::cout writes to the screen.
- Declare, read, compute, print — this exact pattern is most of every program you write this semester.

SEE YOU NEXT WEEK

0.1 + 0.2 is not 0.3

```
double a = 0.1;
double b = 0.2;

if (a + b == 0.3)
    std::cout << "equal";
else
    std::cout << "NOT equal";
```



- Same pattern as the last slide: declare, compute, print.
- Run it. The == check prints “NOT equal.” Every single time.
- Nothing here is broken — next week, why floating-point numbers do this on purpose.

RIGHT AFTER THIS

Lab 0: prove your toolchain works

1

A green build

```
cmake --preset debug && cmake --build --preset debug
```

2

All tests passing

```
ctest --preset debug
```

3

Debugger at a breakpoint

Stop on the line marked <<< BREAKPOINT HERE

4

A sanitizer report

Caused on purpose — then put the comment back

RECAP

What to carry into every week after this

- “Undefined reference” is a linker problem — not a typo, and not a compiler bug.
- Header guards, taught properly in week 5, prevent the double-paste #include creates without them.
- argv — a char** you already used — is exactly what pointer arithmetic operates on in weeks 7–8.
- Every project this semester is graded under the strict preset. Zero warnings is not optional politeness.

Next week

How values are actually stored

Types, sizeof, overflow, and why == on doubles is a trap. See you in Lab 0.